AI-Native CRM — Admin Configuration Guide

Configure workflows, tailor RBAC to your preference, and turn functions on/off at will

Audience: tenant administrators (Super Admin / CRM Admin).
What this guide covers — exactly the three things you asked for:

1. **Activate / deactivate functions you don't need** — turn whole modules on or off (and finer on/off switches).
2. **Configure RBAC to your preference** — design your own roles and decide precisely what each can see and do.
3. **Configure workflows** — automate the CRM with triggers → conditions → actions, and activate/deactivate them at will.

Each part shows the **point-and-click (UI)** path *and* the **underlying API** (for power users / scripting), the **permission** required, and notes that **every change is recorded in the audit log**.

Before you start

What you need	Detail
An admin login	On the live demo: **Tenant** `apar-elite`, **Email** `demo.admin@apar-elite.com`, **Password** `AparAdmin@2026!` (Super Admin — has every permission below).
Where config lives	The **Administration** area (`/admin`) for modules & RBAC, and the **Workflows** area (`/workflows`). The left nav only shows these if your role grants `admin.*` / `workflows.*`.
API base	All endpoints are under `/api/v1`. Send `Authorization: Bearer <accessToken>` from login.
Safety net	Most changes are **reversible** (toggle back on, re-enable a role/permission). Sensitive events are **audit-logged** (who changed what, when). Module/role changes take effect on the user's next page load / sign-in.

> **The golden rule of this platform: nothing is hard-coded to your org.** Modules, role permissions, labels, option lists, and automations are all data you control — change them whenever your needs change.

Part 1 — Activate / deactivate functions at will

You can switch capability on and off at several levels, from an entire module down to a single field or automation. Use the **lightest** switch that achieves what you want.

1.1 Turn an entire module on or off

Disabling a module removes it from **everyone's** navigation in your tenant (its data is retained, not deleted — re-enable any time).

UI

1. Go to **Administration** → **Modules** (`/admin/modules`).
2. You'll see all **23 modules** with a toggle each.
3. Flip the toggle for any module you don't need (e.g. turn off *Social*, *Resellers*, or *Training* if your org doesn't use them).
4. **Save.** Disabled modules immediately disappear from users' left navigation; the change is audit-logged.

API

```
GET  /api/v1/tenant-config/modules
# current on/off state (needs admin.view)
PUT  /api/v1/tenant-config/modules
# set state (needs admin.edit or admin.configure)
     { "modules": [ { "moduleKey": "social",
"enabled": false },
                    { "moduleKey": "resellers",
"enabled": false } ] }
```

The modules you can toggle (23)

leads	accounts	contacts	opportun
campaigns	social	marketing	sales
business_development	presales	partners	resellers
support	customer_success	training	customer
customer_portal	dashboards	notifications	approvals
workflows	ai	admin	

> Tip: keep `admin` enabled — it's the screen you configure everything else from.

1.2 Finer-grained on/off switches

Switch off…	Where	Effect
A capability for a role	`/admin/rbac` (Part 2)	Removes one action (e.g. *export*, *delete*) from a role without hiding the whole module
A custom field	`/admin/custom-fields` → set *Active* off	Hides a field on forms without deleting its data
An option value (status/stage/source)	`/admin/*` option sets	Retire a choice (e.g. an old lead status) while keeping history
A single workflow	`/workflows` → set status *Inactive*	Pauses one automation without deleting it (Part 3)

Part 2 — Configure RBAC to your preference

RBAC (Role-Based Access Control) lets you **design your own roles** and decide exactly what each role can see and do. The 28 seeded roles are a starting point — rename them, clone them, restrict them, or build your own from scratch.

2.1 How permissions work

A permission is a **module + action** pair written `module.action` (e.g. `leads.view`, `opportunities.delete`, `workflows.manage_workflow`). A role is simply a **named bundle of these permission codes**. The UI **adapts to whatever you grant** — if a role lacks `campaigns.view`, that user never sees Campaigns; if it lacks `accounts.export`, the Export button is hidden and the API returns `403`.

The 13 actions you can grant per module

Action	Lets the user…
`view`	Open and read records in the module
`view_dashboard`	See the module's dashboard/analytics
`create`	Add new records
`edit`	Change existing records
`delete`	Archive (soft-delete) records
`assign`	Assign ownership / reassign work
`approve`	Approve items in approval flows
`export`	Export data out of the module
`import`	Bulk-import data
`configure`	Administer the module's settings
`use_ai`	Use AI assist within the module
`manage_ai`	Govern AI (prompts/agents/actions)
`manage_workflow`	Create/run automations for the module

These apply across the same **23 module keys** listed in Part 1.

2.2 Create a role and set its permissions

UI

1. Go to **Administration** → **Roles & Permissions** (`/admin/rbac`).
2. Click **Create role**; give it a **name**, **slug**, and **description** (e.g. "Regional Sales — read only").
3. Tick the exact **module + action** permissions you want it to have.
4. **Save** — the role is now assignable.

API

```
GET  /api/v1/rbac/catalog
# every grantable permission code (admin.view)
GET  /api/v1/rbac/roles
# list roles
POST /api/v1/rbac/roles
# create (admin.create / admin.configure)
     { "name": "Regional Sales — read only", "slug":
"regional-sales-ro",
       "description": "View-only sales access",
       "permissionCodes":
["leads.view","accounts.view","opportunities.view","dash
}
PATCH  /api/v1/rbac/roles/:roleId
# rename / re-describe (admin.edit)
PUT    /api/v1/rbac/roles/:roleId/permissions
```

```
# replace the role's permission set (admin.assign)
     { "permissionCodes":
["leads.view","leads.edit","accounts.view"] }
DELETE /api/v1/rbac/roles/:roleId
# remove a role (admin.delete)
```

2.3 Assign roles to people

UI: in `/admin/rbac`, open the **Users** view, pick a user, tick the role(s) they should hold, and save. A user can hold **multiple roles** — their effective permissions are the union.

API

```
GET /api/v1/rbac/users
# users + current roles
PUT /api/v1/rbac/users/:userId/roles
# set a user's roles (admin.assign / admin.configure)
    { "roleIds": ["<role-uuid-1>", "<role-uuid-2>"] }
```

2.4 RBAC tips

- **Least privilege:** start from the narrowest role that does the job; add actions as needed.
- **Try it instantly:** sign in as any of the 28 demo personas (password `AparDemo@2026!`) to *see* RBAC — the nav and buttons change per role.
- **Everything is audited:** role creation, permission changes, and assignments are logged.

Part 3 — Configure workflows (automation)

A workflow is: **a trigger** (when something happens) → **optional conditions** (only if…) → **one or more ordered actions** (do this). Each workflow has a **status** you control: `draft`, `active`, or `inactive` — that's your activate/deactivate switch.

3.1 Build a workflow

UI

1. Go to **Workflows** (`/workflows`) and click **New workflow**.
2. Name it, pick the **module** it applies to, and choose a **trigger** (see the catalogue below).
3. Add **conditions** (field + operator + value) to narrow when it fires.
4. Add one or more **actions**, set their **order** (sequence), and configure each.

5. Save as **Draft** while you build; flip to **Active** when ready (3.3).

API

```
GET  /api/v1/workflows/catalog
# all trigger / action types
POST /api/v1/workflows
# create (workflows.create / configure /
manage_workflow)
     { "name": "Hot lead follow-up", "module":
"leads",
       "triggerType": "ai_score_changed",
       "conditions": [ { "field": "score",
"operator": "gte", "value": 80 } ] }
POST /api/v1/workflows/:id/actions
# add an ordered action (workflows.edit /
manage_workflow)
     { "actionType": "send_notification", "sequence":
1,
       "actionConfig": { "to": "owner", "message":
"Hot lead — follow up today" } }
PATCH  /api/v1/workflows/:id/actions/:actionId
# edit / enable / reorder an action
DELETE /api/v1/workflows/:id/actions/:actionId
# remove an action
```

3.2 Trigger & action catalogue

Triggers (14) — "when this happens"

record_created	record_updated	stage_changed
assignment_changed	date_reached	sla_breached
campaign_response_received	ticket_escalated	ai_score_chang
customer_health_changed	onboarding_delayed	training_incomp
renewal_approaching	usage_dropped	

Actions (14) — "do this"

assign_owner	create_task	send_nc
send_email	update_field	change_
trigger_approval	call_webhook	run_ai_p
run_ai_agent	create_support_ticket	assign_t
create_customer_success_task	trigger_renewal_playbook	

Condition operators (9): `eq` , `ne` , `gt` , `lt` , `gte` , `lte` , `contains` , `exists` , `in` .

3.3 Activate / deactivate a workflow at will

This is the on/off switch for a single automation — no deletion needed.

UI: open the workflow in `/workflows` and set its **status**: **Active** (runs on its trigger) or **Inactive** (paused). Keep it **Draft** while editing.

API

```
PATCH /api/v1/workflows/:id
# (workflows.edit / configure / manage_workflow)
      { "status": "active" }      # turn on
      { "status": "inactive" }    # turn off (pause)
      { "isEnabled": false }      # alternative
enable flag
```

You can also **enable/disable individual actions** inside a workflow with `isEnabled` on the action — useful to test one step at a time.

3.4 Test & audit

```
POST /api/v1/workflows/:id/run
# run manually with a test context
(workflows.manage_workflow)
     { "context": { "score": 92 } }
GET  /api/v1/workflows/:id/runs
# run history (success/failure per step)
GET  /api/v1/workflows/runs/:runId
# one run's detail
```

Run history records each execution and per-action outcome, so you can confirm a new automation behaves before activating it broadly. Sensitive actions (e.g. `trigger_approval`, AI actions) route through the **Approvals** and **AI governance** flows.

Example — "Hot lead, act fast": Trigger `ai_score_changed` on *leads* → condition `score gte 80` → action 1 `send_notification` to the owner → action 2 `create_task` "Call within 1 hour". Save as draft, **Run** with a test context to verify, then set status **Active**.

Permissions quick reference

To do this	You need (any one)
See admin/config screens	`admin.view` or `admin.configure`
Enable/disable modules, edit theme/terminology/settings	`admin.edit` or `admin.configure`
Create a role / custom field	`admin.create` or `admin.configure`
Set a role's permissions / assign roles to users	`admin.assign` or `admin.configure`
Delete a role / custom field	`admin.delete` or `admin.configure`
Create a workflow	`workflows.create`, `workflows.configure`, or `workflows.manage_workflow`
Edit / activate / deactivate a workflow	`workflows.edit`, `workflows.configure`, or `workflows.manage_workflow`
Run a workflow manually	`workflows.manage_workflow`, `workflows.configure`, or `workflows.edit`

The **Super Admin** demo login (`demo.admin@apar-elite.com`) holds all of these.

Troubleshooting

Symptom	Cause	Fix
Admin or Workflows missing from nav	Your role lacks `admin.*` / `workflows.*`, or the module is disabled	Sign in as Super Admin; re-enable the module
A change didn't appear for a user	Effective on next page load / sign-in	Have them refresh or re-login
`403 / Unauthorized` on an API call	Token's role lacks the permission in the table above	Grant the permission or use an admin token
A workflow never fires	Status is `draft` / `inactive`, or conditions exclude the record	Set status `active`; re-check condition field/operator/value

*Companion documents: the **Module Guide** (what each module does) and the **Persona-Wise Workflow Guide** (per-role tasks). Source of record: the live `rbac`, `tenant-config`, and `workflows` APIs under `/api/v1`.*